IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 3 December 2025, accepted 21 December 2025, date of publication 24 December 2025, date of current version 7 January 2026.

Digital Object Identifier 10.1109/ACCESS.2025.3648134

SURVEY

Optimization in Information Retrieval: A Systematic Review of Techniques for Performance and Scalability

ANDRÉ TOMITAN BOCCES[1], ALEXANDRO BALDASSIN[1], DANIEL CARLOS GUIMARÃES PEDRONETTE[1], AND ALLBERSON BRUNO DE OLIVEIRA DANTAS[2], (Member, IEEE)

[1]Instituto de Geociências e Ciências Exatas (IGCE), Universidade Estadual Paulista Júlio de Mesquita Filho (UNESP), Rio Claro, São Paulo 13506-900, Brazil
[2]Universidade da Integração Internacional da Lusofonia Afro-Brasileira (UNILAB), Redenção 62790-000, Brazil

Corresponding author: André Tomitan Bocces (at.bocces@unesp.br)

This work was supported by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001 [Grant ROR ID: 00x0ma614].

ABSTRACT Information Retrieval (IR) systems are fundamental across various domains, from web search engines to enterprise data management, enabling efficient access to relevant information within large-scale digital collections. While research in IR has traditionally prioritized retrieval effectiveness, ensuring the efficiency and scalability of these systems is equally critical, particularly as data volumes and computational demands continue to rise. Over the past decades, substantial progress has been made in optimizing IR systems, with studies addressing indexing strategies, parallelization techniques, and resource-efficient retrieval. However, given the growing need for real-time and large-scale processing, further advancements in optimization techniques remain essential. This systematic review explores key methods that enhance IR performance through memory management, query execution improvements, and algorithmic efficiency. By synthesizing research from 2018 to 2024, we highlight approaches that significantly improve response time, resource utilization, and scalability in IR applications. Our goal is to provide researchers and practitioners with a comprehensive overview of optimization strategies in IR, emphasizing their critical role in modern retrieval systems. While optimization research may not always receive the same level of attention as other IR topics, it remains a foundational aspect of building high-performance, scalable search technologies. This review underscores the ongoing efforts in this area and identifies opportunities for further innovation.

INDEX TERMS Information retrieval, optimization techniques, systematic review, performance enhancement, scalability, high-performance computing, GPU acceleration, parallel processing.

I. INTRODUCTION

Information Retrieval (IR) is a fundamental field of Computer Science, applied in modern search applications [1], [2]. IR systems are designed to efficiently search and rank documents by assessing their relevance using similarity metrics. As data collections grow exponentially, both effectiveness and efficiency have become crucial for retrieval systems [3], [4]. However, optimizing IR algorithms is a complex challenge that requires multiple techniques to enhance retrieval steps.Optimization, in this scenario, refers to the systematic improvement of four key metrics: *Response Time*, which is how long a query runs for, *Throughput*, the amount of queries processed per unit time, *Memory Usage* to measure the efficiency of RAM or GPU memory footprints, and lastly *Scalability*, that is the ability to maintain performance under increasing data volumes or query loads. Better optimization directly translates to improved values across these metrics. For example, inefficiencies in data transfer between processors and memory can introduce

The associate editor coordinating the review of this manuscript and approving it for publication was Qiang Li.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

high latency, affecting system performance as datasets grow larger [5]. This systematic review aims to consolidate recent advances in IR optimization techniques, analyzing how various approaches improve retrieval performance while addressing scalability and resource efficiency, following the methodology outlined by Carrera-Rivera et al. [6].

A significant portion of IR research focuses on developing new methodologies that improve retrieval accuracy and functionality, emphasizing the novelty and correctness of algorithms, and pushing the boundaries of both theoretical and applied IR [7]. Optimization is an integral part of the development of IR systems, but its study often plays a complementary role to other research priorities. For instance, unsupervised learning methods have contributed significantly to image retrieval [8], but many of these methods require high computational power, highlighting the ongoing challenge of balancing accuracy with efficiency for large-scale applications.

Figure 1 shows the ten most frequent keywords of the 200 most relevant IR journals in 2024. The dominant themes include learning models, natural language processing, and semantics. Notice how terms like "efficiency" and "optimization" do not appear among the most cited keywords, reflecting the distinct research focus on methodological innovation.

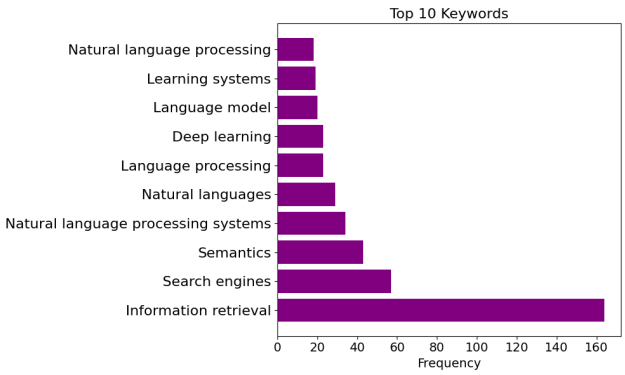

FIGURE 1. Top 10 keywords from the top 200 most relevant journals from 2024 in IR.

Although the introduction of new models and algorithms often attracts more attention in academic publications, optimization is an essential component to translate theoretical advances into practical, high-performance IR solutions. The growing need for real-time processing and large-scale retrieval underscores the importance of refining existing methods to improve efficiency. For instance, Valem and Pedronette [9] emphasize that achieving optimal retrieval precision requires careful balancing of computational resources, particularly in parallel and distributed environments.

Optimization remains a vital component in ensuring that theoretical advances translate into effective, real-world applications, even though it may not always be the primary focus of IR research. Therefore, this review systematically examines optimization techniques that improve response time, memory efficiency, and overall system scalability in IR. Recent contributions in this field of research highlight how performance-oriented strategies can complement algorithmic and model-driven innovations. Serving as a bridge between the fields of Information Retrieval and systems-level optimization, this systematic review offers researchers and practitioners a comprehensive perspective on how efficiency can enhance the practical deployment of retrieval systems at scale.

A. SCOPE OF THIS REVIEW

This review focuses specifically on literature from 2018 to 2024 to ensure this review serves as a practical reference for modern IR challenges. This scope prioritizes recent optimization advancements that address the growing need for real-time processing and large-scale retrieval, rather than providing a historical account of the field. While foundational works like WAND [10] and WAVES [11] remain critical, as well as advancements in query optimization, algorithm selection, and distributed IR, our analysis centers on how optimization has been approached in the context of recent technological trends, including neural representations and the increasing use of embeddings within IR pipelines. This is in line with the generic architecture presented in the review, which prominently features embeddings and related components.

The filtering criteria and inclusion methodology, thoroughly detailed in the Methodology section, strictly define the set of works considered. All analyses, discussions, and identified trends are therefore based solely on the set of articles that passed these inclusion criteria. This methodological constraint ensures reproducibility and transparency, but also inherently limits the review's historical breadth. However, this choice enables a clearer and more focused narrative centered on recent innovations and synergies between modern optimization techniques in IR. Importantly, this work does not seek to propose new theoretical contributions or algorithms. Its primary goal is to act as a structured gateway to the most recent literature in the area, highlighting how current research tackles efficiency issues in IR systems. In this sense, the review may also reveal potential gaps in the literature or emerging patterns, acting as a support tool for future research directions rather than a comprehensive historical account.

B. RELATED WORKS AND CONTRIBUTIONS

Recent systematic reviews in Information Retrieval have primarily concentrated on effectiveness-centric approaches. Guo et al. [12] provided a comprehensive survey of neural ranking models, analyzing their underlying assumptions and design principles from the perspective of retrieval accuracy. Lin et al. [13] surveyed pretrained transformers for text ranking, focusing on BERT-based approaches and their effectiveness in multi-stage ranking architectures. More recently, Wang et al. [14] examined BERT-based information

retrieval methods, organizing techniques by their approach to handling long documents, integrating semantic information, and query expansion. Zhao et al. [15] systematically reviewed dense text retrieval methods based on pretrained language models, categorizing work by architecture, training strategies, and integration approaches. Peikos et al. [16] provided a systematic review of multidimensional relevance estimation, examining how IR systems incorporate multiple relevance factors beyond topical similarity.

While these surveys have made substantial contributions to understanding retrieval effectiveness and algorithmic innovations, they predominantly address *what makes retrieval more accurate* rather than *what makes it faster and more scalable*. In contrast, our review uniquely focuses on performance optimization from a systems-level perspective, examining techniques that enhance response time, memory efficiency, and computational scalability. This complementary viewpoint bridges the gap between IR methodology research and high-performance computing, addressing the practical implementation challenges that arise when deploying modern IR systems at scale. Our emphasis on recent literature (2018–2024) captures how optimization intersects with contemporary trends such as neural representations and embedding-based retrieval, which have become central to modern IR pipelines as illustrated in our generic architecture (Figure 2). By adopting this scoped approach, the review remains manageable, relevant, and useful to a contemporary audience, guiding researchers who are interested in optimizing IR systems in line with recent developments, especially in scenarios where embeddings and machine-learned representations are central components of the retrieval process.

The exponential growth of digital data in recent years has made IR systems indispensable across web search engines, digital libraries, and enterprise data management. Ensuring relevant information is retrieved promptly and accurately presents a growing challenge as data volumes increase [17], [18]. By improving performance across different stages of the IR pipeline (including data indexing, query processing, and result ranking) optimization techniques enhance response time and memory usage of retrieval systems [19], [20].

II. A GENERIC INFORMATION RETRIEVAL TASK PIPELINE

The main stages of a generic pipeline used in Information Retrieval (IR) systems are described below, based on the flowchart shown in Figure 2. Each stage is essential for the proper functioning of an IR system, from ingesting diverse data types to selecting and ranking the most relevant results [17], [21], [22]. This generic pipeline applies to a broad range of data modalities, including text, images, audio, video, and structured data sources.

A. DATA SOURCE

This refers to the collection of information that the IR system will process. Data can originate from multiple modalities, such as structured databases (e.g., relational tables), unstructured text (e.g., web content, logs), multimedia (e.g., images, videos), and even sensor-generated data. The variety and complexity of data sources require the IR system to adapt its ingestion methods to handle diverse formats effectively and ensure uniform processing;

B. PREPROCESSING

During preprocessing, raw input data is transformed into standardized representations suitable for downstream tasks. For *text*, this includes steps such as tokenization, stopword removal, or text normalization. For *images*, preprocessing might involve resizing, noise removal, or feature extraction using techniques such as edge detection or convolutional filtering. For *audio and video*, preprocessing can include noise suppression, segmentation, or spectral transformations. This stage ensures that different data types are efficiently prepared for indexing, embedding, or feature representation. For instance, the choice of embedding type to represent the documents can add to the complexity of preprocessing, as different embeddings may require specialized handling to capture semantic depth;

C. INDEXING

Indexing organizes the preprocessed data into an efficient structure for retrieval. For *text-based IR*, this often involves inverted indexes and mapping terms to document identifiers. In multimedia IR, indexing may use feature-based data structures, such as visual descriptors for images or audio spectrogram representations. For structured data, indexing techniques like B-Trees or hash maps enable efficient querying. The goal is to facilitate rapid lookup in posterior stages while maintaining scalability across large datasets;

D. EMBEDDING COLLECTION

After indexing, the data can be represented using embeddings or feature vectors that capture the semantic or structural characteristics of the content. For text, this might involve word embeddings (e.g., Word2Vec, GloVe) or sentence-level embeddings [23]. In image retrieval, feature vectors extracted from deep learning models (e.g., ResNet, VGG) are commonly used [24]. For audio and video, embeddings can represent waveform characteristics or temporal patterns. Further steps for similarity computation, sorting, and ranking are only possible by representing documents in a standardized dense format, ensuring consistency across the retrieval pipeline;

E. RAW QUERY AND PROCESSED QUERY

A user's query can be expressed in various modalities, such as text input, image uploads, or voice commands. The system preprocesses this query to ensure compatibility with the indexed data. For example, a raw image query might be transformed into a feature vector using a convolutional neural network (CNN), while a spoken query undergoes speech-to-text conversion and subsequent embedding generation. This

processing standardizes the query format, transforming it into a compatible vectorized representation, which enables effective similarity comparisons with the pre-embedded documents (for more advanced forms of query processing, see [25]);

F. SIMILARITY COMPUTATION AND RESULT SELECTION

Once the query has been processed and transformed into an embedding, the system computes its similarity to each of the n documents within the embedding collection. This step measures the closeness between the query and document vectors, using classical similarity metrics such as cosine similarity for text retrieval, which examines the angle between vectors, or Euclidean distance typically used in image retrieval, which calculates the direct distance between points in a high-dimensional space. Advanced similarity techniques, including neural-based similarity enable more accurate cross-modal retrieval (see Shah et al. [26] for a neural-based similarity method applied to e-commerce) or weighted vector comparisons (the study by Shahmirzadi et al. [27] explains multiple term representation approaches commonly used in text similarity computation tasks), can also be used to capture nuanced semantic relationships, enhancing the accuracy of document selection by finding those that align most closely with the user's intent;

G. RANKING AND RE-RANKING

Following the initial selection of relevant documents, the system orders these results based on their computed relevance scores, producing a ranked list where the highest-ranked documents are those most closely aligned with the query. This ranking assigns a score to each document according to the similarity calculation. Subsequently, re-ranking techniques, such as machine learning-based ranking or heuristic adjustments, can be applied to refine this order. These methods consider additional factors, such as document freshness, popularity, or contextual relevance, to enhance the precision of the retrieval, resulting in a more customized and accurate response for the user;

H. ORDERED RELEVANT RESULTS

In the final stage, the system presents the documents to the user as an ordered list based on their calculated relevance to the query. This organized ranked list allows users to quickly and efficiently access the most pertinent information. By structuring results in descending order of relevance, the system prioritizes documents likely to meet the user's needs, streamlines information retrieval, and improves user satisfaction in general with the search experience.

III. METHODOLOGY

This section outlines the systematic review process employed to identify and analyze the relevant literature on optimizing IR systems. The review was conducted following the protocol described in Carrera-Rivera et al. [6]. By using a carefully constructed query, three key aspects were targeted:

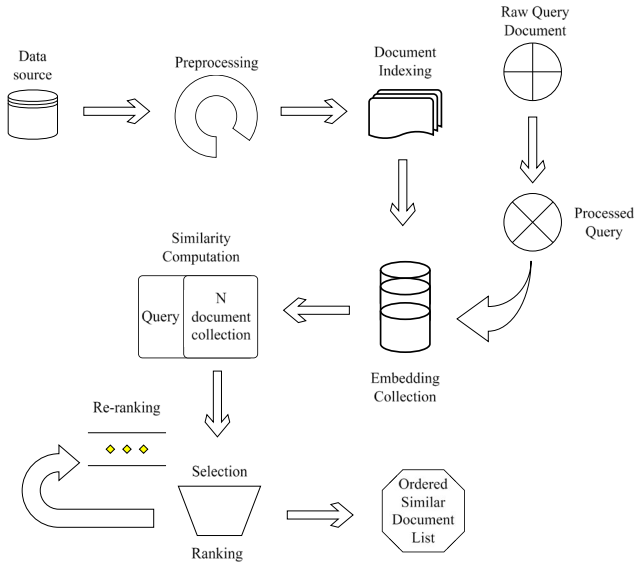

FIGURE 2. A generic information retrieval task pipeline.

TABLE 1. Research questions.

- **What** Optimization Techniques Are **Currently** Used in Information Retrieval Algorithms to Improve Performance?
- **What** Are the **Effects** of Optimization Techniques on Improving Performance in Information Retrieval Algorithms?
- **How** Do Optimization Techniques **Compare** to Traditional or Non-Optimized Techniques in Terms of Performance Improvement in Information Retrieval Algorithms?
- **What** Are the **Factors** that Influence the Effectiveness of Optimization Techniques in Information Retrieval Algorithms?
- **What** Are the **Challenges** in Implementing Optimization Techniques in Information Retrieval Algorithms?

performance gains, optimization techniques, and the field of IR. The query was applied in three major digital libraries, **ACM Digital Library**, **IEEE Xplore**, and **Scopus**, yielding an initial set of 404 documents. These were further filtered according to inclusion and exclusion criteria (see Table 2), allowing the selection to be made to the 20 most relevant and high-quality studies. Using the Carrera-Rivera et al. [6] protocol, several research questions were formulated to guide the understanding of optimizing common stages in IR algorithms (see Table 1). The articles' contents are thoroughly analyzed in a later section regarding the main research questions of this systematic review. This systematic approach ensures a comprehensive and credible foundation for the subsequent analysis of optimization strategies in IR systems.

A. MAIN QUERY

The following query was made by logically combining the three main aspects of the theme of this work using the boolean *AND* clause:

1) **Performance Gains**: filter for articles with performance improvements;

TABLE 2. Inclusion and exclusion criteria.

Inclusion Criteria
• Relevant to the field of Information Retrieval • Focus on Optimization Techniques • Practical contributions/available implementations • Articles with high methodology quality • Recently published (2018-2024)
Exclusion Criteria
• Distant to the field of Information Retrieval • Absent discussion about Optimization Techniques • Articles with little to no data or unavailable implementations • Studies with a lack of applied context • Outdated publications

TABLE 3. Quantitative query results from articles' databases.

• **Scopus:** 196 records • **ACM DL:** 62 articles (filtered for *"artifacts available"* stamp) • **IEEE Xplore:** 146 publications

2) **Optimization Techniques**: filter for articles that explain methods for optimizing tasks;
3) **Information Retrieval**: filter for articles in the field of IR;

Query:

((''efficien*'' OR ''optimiz*'') AND (''Persistent Memory'' OR ''parallel computing'' OR ''multi-threading'' OR ''speculation techniques'' OR ''transactional memory'' OR ''prefetching'' OR ''OpenMP'') AND (''information retrieval'' OR ''IR'' OR ''search engine'' OR ''document retrieval'' OR ''text retrieval'' OR ''ranking algorithms''))

Notice how the boolean *OR* clause expands the scope within a theme, while the *AND* clause potentially narrows filtering, the *OR* clause also groups similar keywords to represent a main theme.

In informal phrasing, the main query can be represented as *search for articles about optimization techniques in Information Retrieval*.

B. SUMMARY OF QUERY RESULTS

The query results from the three digital libraries were consolidated and imported into Zotero reference management software, producing 404 documents. Table 3 shows the distribution of the 404 documents by source article database. An additional filter was applied to articles retrieved from the ACM Digital Library, requiring the ''artifacts available'' stamp. This designation indicates that the article provides verified supplementary resources, such as source code repositories, ensuring reproducibility and methodological transparency. This filtering criterion was specific to the ACM Digital Library and was not implemented for Scopus or IEEE Xplore results.

The initial dataset obtained from these searches provides a substantial basis for further refinement according to the predefined inclusion and exclusion criteria (Table 2). Before applying the inclusion and exclusion criteria, duplicates, invalid entries, and irrelevant works were excluded, ensuring a refined pool of literature. The inclusion criteria aimed to select a manageable and representative dataset of articles for an in-depth review, ensuring relevance to the theme and sufficient methodological quality. Ultimately, the curation process improves the validity and reliability of the review.

The complete filtering process is as follows:

1) Utilize Zotero's reference management software to remove duplicates and invalid entries;
2) Apply keyword filtering to narrow down records based on relevance in their abstracts;
3) Conduct a thorough abstract screening using inclusion and exclusion criteria;
4) Perform a full-text assessment to determine relevance based on predefined criteria;
5) Final selection consists of 20 articles that were considered the most relevant publications for in-depth analysis.

C. LIMITATIONS

This systematic review has acknowledged limitions to be discussed. Restricting sources to three digital libraries and a 2018-2024 timeframe can lead to *Selection Bias*, which possibly excludes relevant older foundational works or non-indexed research. We mitigated this through comprehensive database coverage across multiple platforms, though some relevant works outside our scope may have been missed. *Publication Bias* is another concern, as studies with positive optimization results are more likely to be published, potentially overrepresenting technique effectiveness. We attempted to address this by including studies with null findings and implementation challenges, though this bias cannot be fully eliminated.

Another limitation is the *Heterogeneity of Application Domains*. Many optimization techniques demonstrate effectiveness in specific contexts (e.g., web search, text retrieval) that may not transfer to other domains (e.g., multimedia, specialized databases). For instance, GPU-based acceleration techniques [28] show impressive speedups in large-scale text processing but may offer limited benefits for sparse, high-dimensional multimedia indices where memory access patterns differ substantially. Similarly, distributed processing frameworks like Apache Spark [29] provide scalability advantages for general-purpose computing clusters but may not address specialized requirements of real-time IR systems with strict latency constraints. This domain-specific heterogeneity means that optimization techniques cannot be universally evaluated or compared, creating challenges for developing comprehensive optimization frameworks that work across diverse IR applications.

Finally, the *Evolving Technological Landscape* presents a temporal limitation. The rapid advancement of both hardware (e.g., specialized AI accelerators, persistent memory technologies) and software frameworks creates a moving target

for optimization. Techniques evaluated against 2018-era hardware may appear less effective when compared to modern systems, potentially misrepresenting their current relevance. We addressed this by focusing on recent literature and including emerging technologies, though the rapid evolution of the field means some techniques may become outdated quickly.

IV. DISCUSSION OF MAIN RESEARCH QUESTIONS

The research questions presented in this section were constructed to address the central theme of this study: the optimization of Information Retrieval (IR) systems. Their formulation followed a structured methodology, beginning with the overarching objective of identifying strategies to enhance IR processes. Utilizing the PICOC framework from Carrera-Rivera et al. [6]—Population, Intervention, Comparison, Outcome, and Context—generic question templates were designed to systematically investigate key aspects of optimization in IR. These templates provided a foundation for articulating targeted research questions that align with the objectives of this review.

A. WHAT OPTIMIZATION TECHNIQUES ARE CURRENTLY USED IN INFORMATION RETRIEVAL ALGORITHMS TO IMPROVE PERFORMANCE?

Information Retrieval (IR) algorithms have increasingly incorporated optimization techniques to improve efficiency, scalability, and precision. These methods span various areas, including parallel processing, GPU acceleration, distributed data management, memory optimization, enhanced retrieval pipelines, and advanced embedding techniques. Below, these approaches are categorized and discussed in more detail.

1) PARALLEL PROCESSING AND WORKLOAD DISTRIBUTION

Parallel processing is critical to handling large-scale datasets and reducing computational latency. Topcuoğlu et al. [30] present multi-exponentiation on multicore processors as a method to enhance private IR schemes by leveraging parallel capabilities. Liu et al. [31] highlight dynamic query partitioning in Griffin, which distributes workloads across CPUs and GPUs to effectively mitigate processing bottlenecks. Zhang et al. [32] introduce adaptive parallel algorithms that dynamically adjust to heterogeneous data distributions, improving efficiency for diverse datasets. Wei et al. [33] propose parallel hashing with adjustable batch sizes to optimize memory usage without compromising processing speed. Fazlali et al. [28] explore hybrid parallelism frameworks that combine OpenMP and CUDA, enabling efficient workload distribution for computationally demanding retrieval tasks.

2) GPU-ACCELERATED TECHNIQUES

GPUs play a pivotal role in optimizing computationally intensive IR tasks by enabling high-speed parallel execution. Guo et al. [34] discuss GPUDirect RDMA, which facilitates direct GPU-to-memory communication and significantly reduces latency in distributed systems. Guo et al. [35] highlight GPU parallelization for graph-based similarity searches, optimizing clustering and retrieval operations for large datasets. Lam et al. [36] explore private retrieval systems that utilize GPUs to improve query latency while ensuring secure machine learning inference. Shan et al. [37] propose binary embeddings, which enhance storage and retrieval speeds, making them ideal for large-scale data processing.

3) DISTRIBUTED DATA MANAGEMENT AND LOAD BALANCING

Efficient data distribution and management are essential for maintaining resource efficiency and minimizing latency in distributed environments. Dritsas et al. [29] propose optimized partitioning strategies in Apache Spark to balance workloads across distributed systems, enhancing execution efficiency. Zheng et al. [38] discuss techniques for scaling large knowledge graphs using partitioning and distributed training methods to enable efficient embedding generation for massive datasets. Wang et al. [39] highlight normalized storage models to harmonize data from diverse sources, improving retrieval efficiency and facilitating faster query responses.

4) MEMORY AND CACHE OPTIMIZATION TECHNIQUES

Optimizing memory and cache utilization is essential to manage complex data structures and reduce processing delays. Ni [40] propose cache-efficient algorithms that minimize latency in retrieval tasks by leveraging optimized memory hierarchies. Hofstätter et al. [41] introduce dimensionality reduction techniques, such as PCA and autoencoders, to reduce storage requirements while preserving retrieval accuracy. Lin et al. [42] discuss hybrid lexical-semantic models that improve multilingual retrieval performance by balancing precision and computational efficiency, allowing systems to handle diverse queries effectively.

5) RETRIEVAL PIPELINE AND QUERY OPTIMIZATION

Improvements in retrieval pipelines and query processing have significantly enhanced performance across various IR applications. MacDonald and Tonellotto [43] describe declarative frameworks like PyTerrier, which dynamically adjust retrieval pipelines based on query types and index sizes, optimizing system efficiency. Pisani et al. [44] present iterative re-ranking methods that refine query results, improving relevance without incurring substantial computational overhead. He et al. [45] highlight cross-modal retrieval systems that achieve high accuracy and fast response times, even in environments without GPU support, demonstrating the versatility of these optimization techniques.

6) ADVANCED EMBEDDING AND REPRESENTATION TECHNIQUES

Innovations in embedding techniques continue to drive advancements in retrieval accuracy and scalability.

Cohen et al. [46] propose feed-forward neural networks to replace traditional regression forests, reducing training times while improving ranking performance. Shrestha et al. [47] present multi-vector embeddings, which enhance memory efficiency and response times, especially in large-scale search systems, by providing more robust representations for complex datasets.

7) EMERGING OPTIMIZATION TECHNIQUES

The upcoming quantum computing technologies are promising for the field of IR, as they offer significant improvements in data processing [48]. Quantum computing can enable the handling of large and complex datasets, which comparatively current systems are unable to process in optimal time. Similarly, neuromorphic architectures have proven promising: a study on optimizing nearest-neighbor search, a common IR task, demonstrates how the technology improves scalability [49]. In terms of specialized hardware, Persistent Memory (PM) is a viable candidate for bridging the gap between traditional DRAM and slower storage devices. Using PM for efficient OLAP query processing, Tang et al. [50] presents compelling evidence that the technology is adequate for optimizing IR workloads that heavily rely on memory-bound operations. Nevertheless, there are still complex impeditive integration issues to overcome to use the full potential of the emerging technology in IR applications, and the near future of IR optimization research will be spent resolving the challenges of implementing cohesive quantum IR systems.

8) DISCUSSION

Optimization techniques in Information Retrieval algorithms highlight a versatile approach to enhance computational efficiency, scalability, and accuracy. Central methods include parallel processing and GPU-accelerated techniques, which improve processing speed and workload distribution across multicore and GPU resources, exemplified by strategies like hybrid parallelism [28] and GPUDirect RDMA [34]. Distributed data management methods, such as optimized partitioning in Apache Spark [29] and knowledge graph embeddings [38], address scalability challenges, while memory and cache optimization techniques balance precision and computational efficiency to handle diverse datasets effectively. Retrieval pipeline optimizations, like PyTerrier [43] and iterative re-ranking [44], demonstrate flexibility, and advanced embedding strategies, such as multi-vector embeddings [47], drive improvements in ranking and scalability.

B. WHAT ARE THE EFFECTS OF OPTIMIZATION TECHNIQUES ON IMPROVING PERFORMANCE IN INFORMATION RETRIEVAL ALGORITHMS?

The impact of optimization techniques on IR algorithms is observed primarily in three key areas: faster processing time, scalability, and memory efficiency. This section presents optimization techniques by focusing on prominent themes.

TABLE 4. Table from Steinert et al. [35] shows improved processing time using parallel techniques and GPU acceleration for *Graph Code* calculation and ordering of subsets of the WaPo dataset. Results with and without sort time included.

# GCs	CPU SEQ without Sort [s]	CPU SEQ with Sort [s]	CPU Par without Sort [s]	CPU Par with Sort [s]	CUDA Par without Sort [s]	CUDA Par with Sort [s]
100	0.005	0.005	0.002	0.002	0.003	0.003
1,000	0.045	0.045	0.011	0.011	0.006	0.006
10,000	0.462	0.465	0.100	0.102	0.012	0.013
100,000	4.756	4.787	1.021	1.052	0.079	0.092
250,000	10.421	10.502	2.560	2.642	0.163	0.202
500,000	18.078	18.252	4.917	5.091	—*	—*
728,618	21.942	22.210	7.195	7.462	—*	—*

* out of memory.

Each topic is discussed with references to the relevant works, emphasizing their similarities and differences.

1) PARALLELIZATION TECHNIQUES FOR PERFORMANCE ENHANCEMENT

Parallelization consistently yields significant reductions in query response time. Topcuoğlu et al. [30] applies parallel multiexponentiation to balance workloads across threads in private IR tasks to reduce computational latency. Steinert et al. [35] explores GPU capabilities for accelerating similarity search acceleration in graph clustering, and Figure 4 shows a comparative table for different algorithmic configurations' processing results. Compared to the two previous examples, Zhang et al. [32] propose parallel and adaptable top-K algorithms based on GPU targeting retrieval tasks with algorithms tailored to extract high parallelism from the GPU while considering data distribution to optimize response time.

Further advancements are seen in hybrid systems, such as the work of Liu et al. [31] on adaptive CPU-GPU parallelism in Griffin, which dynamically partitions queries between CPU and GPU, adapting to workload variations according to query characteristics, since the CPU is generally fitter in deep branching tasks, while the GPU excels in SIMD parallel tasks, improving both response times and memory management.

2) EFFICIENT MEMORY MANAGEMENT IN IR SYSTEMS

Techniques like top-K optimized algorithm for memory hierarchies, as seen in the work of Ni [40], focus on selection tasks, utilizing cache layers to improve data access efficiency. Similarly, Shan et al. [37] propose a binary embedding representation for GPU optimization that employs memory-efficient binary embeddings to accelerate retrieval in GPU environments. These approaches emphasize efficient memory use, particularly in large-scale scenarios, but challenges remain in optimizing trade-offs between memory consumption and retrieval accuracy. In ranking tasks, Hofstätter et al. [41] explore the reduction of low-frequency terms using optimized embeddings to reduce memory overhead by optimizing embeddings for rare terms. Although effective for extensive databases, this technique shares a common goal with the work of Cohen et al. [46],

which replaces memory-intensive regression forests with compact feedforward networks in learning-to-rank pipelines.

3) SCALABILITY STRATEGIES FOR LARGE-SCALE INFORMATION RETRIEVAL

Dritsas et al. [29] implement parallel clustering in Apache Spark to optimize clustering tasks by distributing data between nodes in Apache Spark, allowing processing larger data sets. Scalability optimizations also come in the form of improved communication between processing units; the use of GPUDirect RDMA by Guo et al. [34] enhances query processing by optimizing data transfers between CPU and GPU in distributed joins, thus reducing latency for large data operations.

Hierarchical approaches, such as the use of hierarchical embeddings for multi-vector retrieval, as seen in the work of Shrestha et al. [47], enhance scalability in ranking systems by organizing embeddings into efficient multi-vector structures. This complements techniques such as optimized graph partitioning using the DGL-KE library from Zheng et al. [38], which partitions knowledge graphs for parallel embedding training, improving performance in handling vast knowledge bases.

4) DISCUSSION

The techniques discussed above demonstrate various approaches to improving IR performance through optimization. Studies on multicore and GPU parallelization allocate efforts to reduce processing time, while hierarchical memory optimization and scalable methods target efficiency in resource use and adaptability to large datasets, providing overarching scalability improvements in their results.

Nevertheless, strategies often intersect in results, as seen in the work of Shan et al. [37] and Hofstätter et al. [41], which simultaneously improves memory efficiency and retrieval speed. The main synthesis is that the optimization techniques highlighted contribute to enhancing IR system performance. However, while these methods show significant improvements, there remains room for further refinement, particularly in integrating multiple optimization strategies to address evolving computational challenges and dataset complexities.

Future research can continue building on existing advancements by exploring hybrid approaches that integrate multiple optimization strategies—such as combining advanced parallelization with hierarchical memory management and scalable ranking models—to further enhance IR system performance.

C. HOW DO OPTIMIZATION TECHNIQUES COMPARE TO TRADITIONAL OR NON-OPTIMIZED TECHNIQUES IN TERMS OF PERFORMANCE IMPROVEMENT IN INFORMATION RETRIEVAL ALGORITHMS?

Optimization techniques in Information Retrieval (IR) demonstrate substantial advantages over traditional, non-optimized approaches, particularly in improving speed, scalability, memory efficiency, and communication. This section discusses key optimization categories, highlighting their performance gains over traditional techniques while analyzing their similarities, differences, and applicability to various pipeline stages.

TABLE 5. Table taken from Fazlali et al. [28] shows improved processing time using parallel techniques (OpenMP) and GPU acceleration (CUDA) on the 4-similarity problem.

Size	$T_{seq.}$	T_4	T_8	T_{16}	T_{32}	T_{50}	T_{CUDA}
128	0.3001	0.2187	0.1462	0.0758	0.0390	0.0258	**0.0203**
256	4.2894	2.7831	1.7774	0.9331	0.4851	0.3101	**0.2421**
512	65.838	44.5858	27.8171	14.9167	7.8976	5.1297	**4.0138**
1024	1077.262	733.357	441.7463	247.8341	133.1242	83.8182	**72.2119**
Avg.	**286.9223**	**195.2361**	**117.8717**	**65.9399**	**35.3864**	**22.3209**	**19.1220**

1) PARALLEL vs. SEQUENTIAL PROCESSING

Several use cases of parallelization techniques were presented in articles across the IR pipeline stages. For example, in clustering, Apache Spark's parallel K-Means implementation, as examined by Dritsas et al. [29], enhances efficiency through distributed processing. Re-ranking tasks also benefit from parallelization. Optimized unsupervised iterative re-ranking, developed by Pisani et al. [44] employs GPU and APU parallelization to accelerate iterative processes, vastly improving upon traditional sequential re-ranking. Similarly, CUDA and OpenMP parallel implementations, such as those explored by Fazlali et al. [28], achieves superior performance in similarity calculations by distributing tasks across CPU and GPU cores, a clear advantage over sequential and single-threaded execution, with an average speedup of 15 times using CUDA parallelism, as seen in Table 5, taken straight from the original paper.

2) EFFICIENT USE OF MEMORY HIERARCHY

Memory efficiency is crucial for IR systems that manage large-scale data. Optimized hashing and batch processing, presented by Wei et al. [33], outperforms traditional nearest-neighbor methods by improving memory utilization in selection tasks. Hierarchical embeddings, explored by Shrestha et al. [47], reduce memory strain in multi-vector retrieval by organizing embeddings into hierarchical structures, a clear improvement over flat embeddings. Figure 3, taken from the article *ESPN: Memory-Efficient Multi-vector Information Retrieval* by Shrestha et al. [47], compares single query latency on different memory configurations, and ESPN shows superior results in every pipeline stage, while using less memory space.

Other techniques, such as the Top-K optimization for memory hierarchy, developed by Ni [40], exploit cache layers to maximize data access speeds. Similarly, the dimensionality reduction of dense vectors, studied by Lin et al. [42], balances semantic and lexical representations to enhance memory efficiency in retrieval tasks.

3) OPTIMIZED COMMUNICATION

Efficient communication between processing units is vital in data-intensive tasks. For example, GPUDirect RDMA,

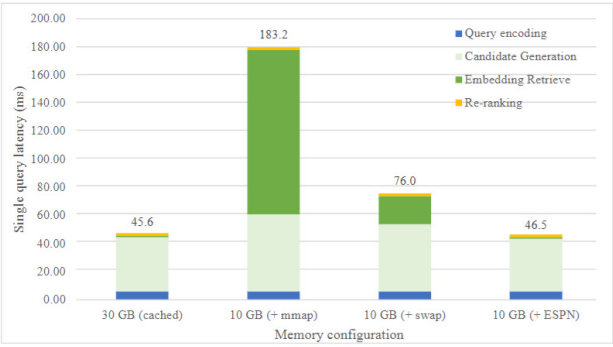

FIGURE 3. Graph comparing memory configurations in terms of single query latency. ESPN (optimized technique) outperforms even the cached configuration.

examined by Guo et al. [34], streamlines data transfer between CPU and GPU, drastically reducing transfer times compared to traditional communication methods. Meanwhile, dynamic query partitioning in Griffin, proposed by Liu et al. [31], minimizes bottlenecks by distributing queries across CPU and GPU in hybrid systems, improving execution times over CPU-only approaches.

4) SCALABLE MODEL TRAINING

Scaling model training to handle large datasets is another area where optimization techniques excel. Feed-forward networks, introduced by Cohen et al. [46], replace regression forests in ranking tasks, achieving faster training with lower memory requirements. Similarly, optimized graph partitioning with DGL-KE, as developed by Zheng et al. [38], enables efficient training for extensive knowledge graphs, outperforming traditional graph training methods. For neural models, low-frequency term reduction, studied by Hofstätter et al. [41], optimizes embeddings of rare terms, reducing training complexity compared to conventional approaches. In cross-modal retrieval, social tags in TLPS, examined by He et al. [45], integrate semantic data to outperform traditional retrieval methods, especially in CPU-only environments. Lastly, optimizing IR pipelines with PyTerrier, demonstrated by MacDonald and Tonellotto [43], enhances indexing and retrieval scalability, providing superior performance over traditional indexing.

5) DISCUSSION

The comparison between optimization techniques and traditional Information Retrieval (IR) approaches reveals a nuanced landscape shaped by application constraints and technological considerations. Optimized methods consistently demonstrate superior performance in speed, scalability, and memory efficiency, but their effectiveness depends on their implementation and adaptability to specific environments. In some cases, traditional approaches may still outperform optimized solutions when factors such as algorithmic complexity, energy efficiency, or hardware constraints are not adequately addressed. For instance, techniques that rely on high-performance hardware, such as GPUs, may not always be viable in resource-constrained settings such as mobile devices or edge computing. In such cases, optimization strategies must adapt to the limitations imposed by the target platform. The TLPS method proposed by He et al. [45] for cross-modal retrieval, which uses social tags, exemplifies this approach. By designing an efficient optimization technique that eschews GPU dependency, TLPS achieves remarkable performance on devices with restricted hardware capabilities, highlighting the flexibility required for practical deployment.

Moreover, incremental advances in optimization frameworks can facilitate transitions from traditional methods to fully optimized systems. By hierarchically integrating advanced techniques on performance bottlenecks, developers can mitigate the implementation challenges associated with adopting cutting-edge solutions. For example, a modular approach to optimization allows IR systems to incorporate GPU acceleration, memory-efficient embeddings, or parallel processing capabilities in stages, ensuring seamless evolution without compromising existing functionality.

Ultimately, the performance gains achieved by optimization techniques depend on their intrinsic efficiency and alignment with the operational context. Tailoring optimizations to the specific demands of each application ensures that their potential is fully realized. By considering practical trade-offs, such as hardware accessibility and deployment constraints, optimization strategies can achieve a balance between theoretical advances and real-world applicability, paving the way for robust and adaptable IR systems.

D. WHAT ARE THE FACTORS THAT INFLUENCE THE EFFECTIVENESS OF OPTIMIZATION TECHNIQUES IN INFORMATION RETRIEVAL ALGORITHMS?

Three key factors shape the effectiveness of optimization techniques in Information Retrieval (IR) algorithms: workload adaptability, computational performance, and data characteristics. These factors influence critical performance metrics such as response time, memory efficiency, and scalability. Their interplay determines the success of optimization strategies across diverse IR scenarios, emphasizing the need for adaptable and context-aware methodologies. This section explores each factor, discussing the relevant works and their contributions.

1) WORKLOAD ADAPTABILITY

Workload adaptability refers to the capability of optimization techniques to dynamically adjust to varying query types, batch sizes, and computational demands across different IR tasks, ensuring consistent efficiency under changing conditions. Techniques that dynamically distribute workloads often achieve better performance by adapting to changing operating conditions. For instance, Griffin employs dynamic query partitioning between CPU and GPU, as demonstrated by Liu et al. [31], optimizing query processing under varying workloads. Similarly, PyTerrier, introduced by MacDonald

and Tonellotto [43], demonstrates pipeline adaptability, enabling adjustments in re-ranking pipelines to accommodate different indexing sizes and query complexities.

In re-ranking tasks, iterative methods distribute workloads effectively across GPUs and CPUs to minimize response times for complex queries, as explored by Pisani et al. [44]. Hybrid approaches, such as those using OpenMP and CUDA, balance similarity calculations across CPU and GPU resources, highlighting the importance of efficient task distribution, as shown by Fazlali et al. [28]. Moreover, techniques optimizing batch sizes in selection tasks, like parallel hashing algorithms, maximize memory and processing efficiency, as described by Wei et al. [33]. These strategies highlight the importance of adaptive workload optimizations in enhancing IR performance across diverse computational environments.

2) COMPUTATIONAL PERFORMANCE AND OPTIMIZATION

The underlying system architecture significantly impacts the success of optimization techniques. Features such as multicore processors, GPU acceleration, and efficient CPU-GPU integration dictate the feasibility of achieving performance gains in embedding, similarity search, and ranking tasks. For example, effective multicore load balancing enhances the performance of parallelized operations, such as multi-exponentiation in private IR systems, as discussed by Topcuoğlu et al. [30]. Similarly, GPUDirect RDMA minimizes latency by facilitating direct data transfers between CPU and GPU, optimizing distributed joins, as shown by Guo et al. [34]. GPU-specific optimizations play a crucial role in similarity search tasks, particularly for large datasets, as seen in GPU-accelerated graph-based techniques demonstrated by Steinert et al. [35]. Binary embedding representations further enhance memory efficiency and retrieval speed, provided the architecture supports high-volume parallel operations, as explored by Shan et al. [37]. For hardware-limited environments, adaptations, such as cross-modal retrieval using social tags, maintain performance even in the absence of GPUs, as described by He et al. [45]. These examples illustrate how architectural capabilities and constraints influence the design and effectiveness of IR optimization methods.

3) DATA CHARACTERISTICS

The structure, size, and complexity of the data that are processed also dictate the effectiveness of optimization techniques. Adapting strategies to specific data characteristics enhances performance in indexing, embedding, clustering, and model training, particularly in large-scale and heterogeneous datasets. As demonstrated in Apache Spark, balanced data distribution is critical in clustering tasks, where efficient load balancing across nodes improves scalability, as shown by Dritsas et al. [29]. Similarly, GPU-based graph processing benefits from larger and more complex datasets, as these scenarios amplify the advantages of parallelization, as noted by Steinert et al. [35].

Knowledge graph partitioning techniques optimize embedding training in multi-GPU systems, ensuring scalability for large datasets, as demonstrated by Zheng et al. [38]. Combining semantic and lexical vectors enhances retrieval accuracy in multilingual embeddings, provided the vector density aligns with dataset characteristics, as discussed by Lin et al. [42]. Hierarchical embeddings, designed for multi-vector retrieval, improve scalability by effectively structuring high-density embeddings, as explored by Shrestha et al. [47]. Furthermore, optimizing term frequency in neural model training reduces memory overheads, enhancing performance for extensive databases, as noted by Hofstätter et al. [41]. These studies emphasize the need for aligning data-specific strategies with optimization objectives, ensuring robustness and efficiency in diverse IR applications.

4) DISCUSSION

The success of optimization techniques in Information Retrieval (IR) largely depends on their ability to handle the complexity and diversity of real-world data. Modern IR systems often operate on heterogeneous [51], unstructured and multilingual [52] datasets, reflecting the ubiquity of web applications and the intrinsic diversity of user-generated content [53]. Multimodal IR systems, which integrate text, images, and other modalities, further amplify these challenges [54]. These characteristics pose unique challenges for optimization, as the techniques must account for significant variability in data structure, linguistic characteristics, and semantic relationships. Similarly, the integration of diverse data modalities—such as text, images, and metadata, requires optimization frameworks to reconcile disparate formats while preserving the precision of the retrieval. This complexity necessitates robust optimization techniques capable of addressing such heterogeneity without compromising performance.

Looking ahead, the dynamic nature of both workloads and computational environments underscores the need for real-time, adaptive optimization frameworks [55], [56]. As IR systems increasingly operate in distributed or cloud-based infrastructures, their performance depends on the ability to respond dynamically to changes in data volume, query patterns, and hardware availability. For example, systems leveraging adaptive query partitioning across CPU and GPU resources, such as Griffin [31], exemplify the potential of dynamic optimization. These frameworks enable IR systems to allocate resources effectively based on real-time demand, optimizing response times, and resource utilization. Future advancements could refine adaptability by integrating predictive analytics and machine learning, enabling proactive workload adjustments and enhancing real-time optimization strategies.

IR systems can achieve greater robustness and efficiency by addressing both data diversity and the need for adaptability. Optimization frameworks that evolve to meet the demands of complex, real-world environments can potentially enhance retrieval performance, ensuring scalability and resilience.

E. WHAT ARE THE CHALLENGES IN IMPLEMENTING OPTIMIZATION TECHNIQUES IN INFORMATION RETRIEVAL ALGORITHMS?

Implementing optimization techniques in Information Retrieval (IR) algorithms involves addressing challenges related to algorithmic complexity, hardware constraints, and infrastructure adjustments. These issues significantly affect the scalability, efficiency, and practicality of deploying such techniques in real-world scenarios. This section provides a detailed discussion of these challenges and highlights relevant works that illustrate obstacles and potential solutions.

1) ALGORITHMIC COMPLEXITY

The inherent complexity of IR algorithms creates obstacles when balancing scalability, memory efficiency, and response time across various tasks, including ranking, embedding, and re-ranking. For instance, Cohen et al. [46] highlight the difficulties in transitioning from regression forests to simpler models such as feed-forward networks in ranking tasks, where preserving accuracy while managing training and parameter tuning is a significant challenge. Similarly, Shrestha et al. [47] demonstrate that implementing hierarchical embeddings for multi-vector retrieval requires careful optimization to balance memory efficiency and response times across datasets of varying sizes.Lin et al. [42] discuss the challenges of combining semantic and lexical vectors in multilingual retrieval, which require fine-tuning to balance linguistic diversity and computational efficiency. Zhang et al. [32] emphasize the need to customize Top-K algorithms to handle heterogeneous data, which requires parallelism and algorithmic behavior to adapt to specific characteristics of the data. Shan et al. [37] explore the trade-offs in using binary embedding representations for similarity search, where maintaining accuracy alongside computational cost is particularly challenging for large and complex queries. Lastly, Hofstätter et al. [41] underscore the need for careful design in dimensionality reduction for classification tasks, where the balance of retrieval quality with processing overhead remains critical in large-scale datasets.

2) HARDWARE CONSTRAINTS

Hardware limitations present significant barriers to the deployment of optimization techniques, particularly in tasks like embedding, retrieval, and indexing. Topcuoğlu et al. [30] address the challenges of achieving efficient load balancing across multicore processors, requiring precise workload distribution to avoid bottlenecks and maximize throughput. Guo et al. [34] show that CPU-GPU integration methods such as GPUDirect RDMA require advanced techniques to reduce latency and improve parallel processing. Steinert et al. [35] highlight memory constraints in GPU-based approaches for graph parallelization in clustering, particularly when processing large and complex graph structures. Lam et al. [36] discuss the added complexity of managing data transfer latency between CPU and GPU in private retrieval systems, where solutions must balance security and performance. He et al. [45] explore alternative strategies, such as leveraging social tags in cross-modal retrieval, which become necessary to maintain efficiency in GPU-limited environments. Finally, Zheng et al. [38] examine the challenges of scaling knowledge graphs for embedding training, highlighting the need for innovative solutions like graph fragmentation and node coordination to address memory and processing constraints.

3) CODEBASE AND INFRASTRUCTURE ADJUSTMENTS

Effective implementation of optimization techniques often requires extensive modifications to the existing codebase and infrastructure, particularly in tasks involving synchronization, data handling, and parallel processing. Liu et al. [31] show that hybrid CPU-GPU systems like Griffin require sophisticated synchronization mechanisms to optimize execution pipelines and minimize latency. Wei et al. [33] discuss the challenges of selecting optimal batch sizes for parallel hashing, which involve balancing memory usage and throughput to enhance overall performance. Dritsas et al. [29] highlight the need for advanced partitioning strategies in distributed systems like Apache Spark to balance workloads and reduce communication latency. MacDonald and Tonellotto [43] emphasize the complexity of adapting IR pipelines in PyTerrier to accommodate diverse query types and index sizes. Fazlali et al. [28] describe the difficulties of synchronizing parallel processing in systems using OpenMP and CUDA, where precise handling of task sizes and workload distribution is essential. Wang et al. [39] underline the importance of advanced data normalization strategies for heterogeneous datasets, particularly in distributed storage systems where effective indexing and retrieval depend on consistent preprocessing.

4) DISCUSSION

The core implementation challenges of IR optimization techniques reside not merely in acquiring the required hardware or computational resources, but in the systemic adjustments necessary to apply such methods to existing IR architectures. Many existing IR systems were built on architectures that lack the modularity or scalability to support modern optimization techniques seamlessly [57]. Retrofitting such systems to incorporate advanced methods can require substantial re-engineering [58]. A gradual restructuring approach, where optimization techniques are incrementally integrated into legacy systems, can help mitigate these issues [59]. For example, modular upgrades, such as adding GPU-based query processing to an otherwise CPU-bound system, allow organizations to adopt advanced optimizations without fully replacing their existing infrastructure.

V. ANALYSIS

A simple flow chart presents the optimization techniques (Figure 4), which serves as a keyword guide for researchers. At the highest level, the diagram differentiates between hardware-based and software-based optimization strategies.

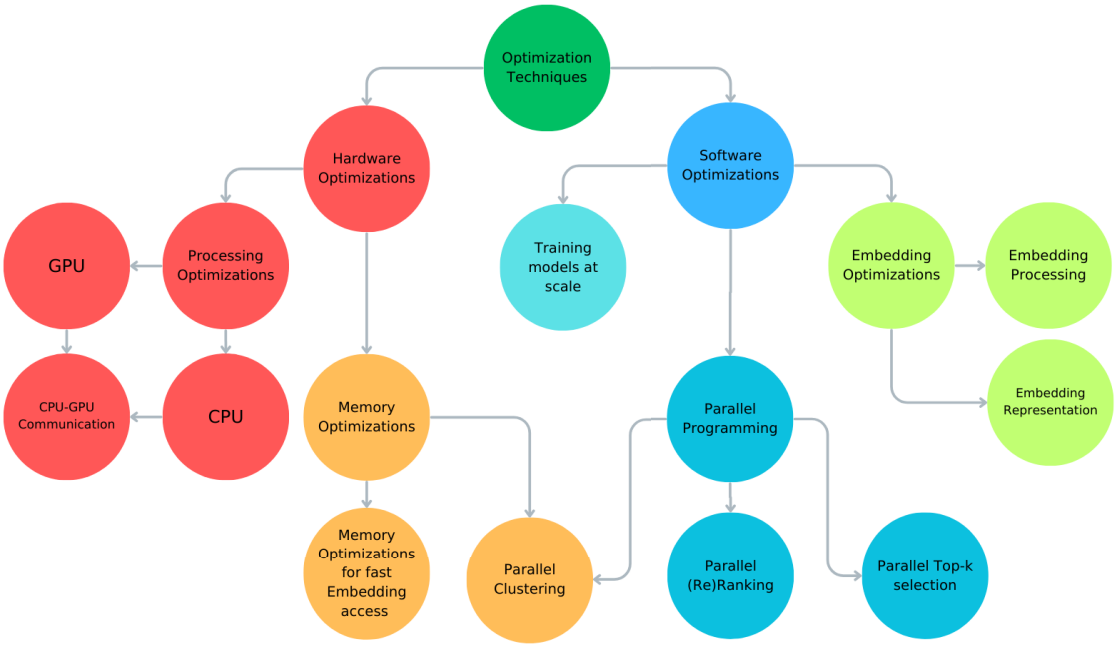

FIGURE 4. An illustration to classify the optimization techniques used in the articles analyzed.

These two categories form the foundation of optimizations of the IR system, with further subdivisions representing more specialized techniques as the graph progresses toward its leaf nodes.

On the hardware side, optimizations are driven by processing and memory enhancements. Processing optimizations are divided between CPU and GPU improvements, focusing on efficient CPU-GPU communication, a critical factor in ensuring that computational workloads are efficiently shared between processors. Memory optimizations, particularly those aimed at enhancing access to embeddings, are essential for managing large-scale data within IR tasks, especially in HPC environments.

On the software side, optimization techniques center around programming strategies and efficient model training. Parallel programming emerges as a key enabler of performance gains, with techniques such as parallel clustering, parallel ranking, and top-k selection widely applied to accelerate IR tasks. Furthermore, embedding optimizations, such as representation and processing improvements, are crucial to ensure that systems can manage and retrieve data more effectively.

Together, these hardware and software optimizations provide a comprehensive approach to improving the efficiency and scalability of IR systems, enabling them to handle the growing volumes of data encountered in modern applications.

A. SUMMARY OF MEASURED GAINS

To provide a quantitative perspective on the effectiveness of these optimization techniques, Table 6 summarizes the performance improvements reported in selected studies. These results highlight the substantial gains achievable through parallelization, GPU acceleration, and memory optimization.

TABLE 6. Summary of performance gains in reviewed studies.

yellow **Study**	**Technique**	**Metric**	**Improvement**
yellow Fazlali et al. [28]	CUDA Parallelism	Speedup	∼15x vs Sequential
yellow Topcuoğlu et al. [30]	Parallel Multi-exp	Speedup	27% (16 threads)
yellow Steinert et al. [35]	GPU Graph Code	Speedup	Up to 225x vs CPU
yellow Guo et al. [34]	GPUDirect RDMA	Speedup	1.83x vs SOTA
yellow Shrestha et al. [47]	ESPN Multi-vector	Speedup	3.9x - 6.4x vs Baseline
yellow Liu et al. [31]	Griffin Hybrid	Speedup	10x vs CPU

B. MOST PROMINENT OPTIMIZATION TECHNIQUES

Through our review of the literature, three optimization techniques were identified as the most prominent. These techniques were evaluated based on their performance impact, scalability, frequency of use, and contribution to the Information Retrieval field. They have shown substantial advancements in improving the efficiency of IR systems, particularly in distributed and high-performance environments.

1) PARALLEL PROCESSING AND GPU UTILIZATION

Parallel processing, particularly on GPUs, has become a cornerstone in optimizing IR algorithms, offering substantial improvements in both speed and scalability [60]. Techniques like parallel Top-K algorithms and intra-query parallelism [32] have demonstrated remarkable reductions in query processing times by distributing computational workloads across many GPU cores. This level of parallelism is especially effective for compute-heavy tasks such as similarity search, ranking, and clustering large datasets. In addition to accelerating individual query responses,

GPU-based parallelization enables IR systems to process a much larger volume of concurrent queries, significantly enhancing system throughput [61]. Using shared memory, registers, and advanced scheduling techniques efficiently, GPUs reduce data transfer bottlenecks and improve memory access patterns, critical in large-scale information retrieval scenarios. Furthermore, modern GPUs support advanced features like mixed-precision computing and tensor cores, which can further accelerate deep learning-based retrieval models and embedding searches. The combined effect is a substantial reduction in latency and improved query response times, making GPUs indispensable for high-performance IR tasks in environments with massive data and strict time constraints.

2) MEMORY OPTIMIZATIONS FOR EFFICIENT EMBEDDING PROCESSING

Memory optimizations are crucial in embedding-based tasks, particularly when dealing with large embedding spaces that require efficient memory management. Approaches such as the ESPN model [47], which employs a sophisticated strategy for storing and retrieving multi-vector embeddings, effectively balance memory usage and response times, significantly reducing latency in large-scale search tasks. Furthermore, hierarchical memory strategies, including optimized data caching and retrieval techniques [40], alleviate memory bottlenecks, especially when managing vast datasets. These methods are particularly relevant when using additional memory layers like Persistent Memory (PM), which offers non-volatile storage with near-RAM speed. The inclusion of PM can bridge the gap between traditional RAM and disk storage, enhancing retrieval performance by maintaining high-speed data access while offloading some of the burden from main memory. Using this layered memory architecture, systems can support larger datasets without compromising speed, leading to greater scalability in memory-intensive IR tasks.

3) HYBRID CPU-GPU SYSTEMS

Hybrid systems that combine the computational capabilities of both CPUs and GPUs provide substantial performance improvements by leveraging the strengths of each architecture. Systems such as Griffin [31] dynamically partition queries between CPUs and GPUs, utilizing the strengths of both processors to optimize resource allocation and minimize bottlenecks. CPUs are particularly well-suited for tasks that require complex branching, task-level parallelism, and fine-grained control, making them ideal for managing control-heavy and diverse operations. On the other hand, GPUs excel in Single Instruction, Multiple Data (SIMD) parallelism, offering massive computational throughput for tasks that involve large-scale data processing, such as matrix operations and other computationally intensive workloads. By combining these capabilities, hybrid systems can significantly enhance real-time query processing, enabling more efficient handling of high query volumes in distributed or high-performance IR systems. This synergy results in improved throughput and reduced latency, especially in applications requiring both flexibility (controlled by the CPU) and computational intensity (accelerated by the GPUs).

C. PRACTICAL IMPLEMENTATION CHALLENGES

The optimization techniques discussed in this review provide substantial benefits in terms of efficiency, scalability, and accuracy, but their adoption can be constrained by practical challenges rather than a lack of awareness or interest from researchers. Implementing advanced IR optimization strategies requires significant investments in infrastructure, expertise, and continuous system maintenance, which can pose serious feasibility concerns, particularly for smaller enterprises or organizations with limited resources.

Many advanced optimization methods for Information Retrieval (IR) systems depend on computationally expensive infrastructures such as GPUs, TPUs, and distributed computing [62]. These systems often require substantial upfront investment and ongoing operational costs, which may not be justifiable for organizations with smaller-scale retrieval tasks [63]. While cloud-based solutions can help reduce hardware costs, they introduce recurring expenses that may accumulate over time, posing financial challenges for long-term sustainability. The other relevant aspect about adopting these optimization techniques is that it often requires skilled professionals in the fields of high-performance computing and distributed computing, which can be costly to hire and retain. Training existing employees in these advanced methods further requires time and resources, diverting focus from core business activities.

In addition, the integration of optimized IR solutions into existing workflows can be complex, particularly when legacy systems are incompatible with modern techniques. This requires significant re-engineering efforts and careful planning to ensure minimal disruption during migration, and although advanced optimizations improve performance, they often introduce trade-offs in terms of cost, complexity, and maintenance [64]. For example, deep learning-based models can enhance retrieval accuracy but at the cost of increased latency and inference expenses, which may be impractical for real-time applications with limited budgets. Smaller enterprises, in particular, face difficulties justifying such investments, especially if their IR needs are modest. As a result, finding cost-effective and scalable alternatives, such as lightweight indexing techniques and hybrid retrieval methods, remains a key challenge for these organizations.

D. STRATEGIES FOR OVERCOMING IMPLEMENTATION CHALLENGES

Given the challenges associated with optimizing IR systems, some strategic approaches can facilitate successful implementation. Proactively addressing these implementation challenges can improve the efficiency of their IR

systems while ensuring that optimizations remain practical and maintainable in real-world applications.

- **Modular Upgrades:** Instead of overhauling the entire system at once, organizations can incrementally update components such as indexing structures, ranking algorithms, or storage solutions while maintaining compatibility with existing frameworks;
- **Hybrid Approaches:** Running optimized methods in parallel with legacy components allows organizations to benchmark improvements before full adoption;
- **Cloud-Based Solutions:** Leveraging cloud computing for resource-intensive optimizations, such as distributed search and neural ranking models, can reduce infrastructure constraints.

E. REAL-WORLD APPLICATIONS OF OPTIMIZATION TECHNIQUES

The practical utility of optimization techniques is evident across diverse IR applications. In e-commerce platforms, neural similarity models [26] handle millions of product matches with high precision while maintaining sub-second response times. Major web search engines employ multi-level caching strategies [65], [66] to serve billions of daily queries with consistently low latency. For resource-constrained environments like mobile devices, cross-modal retrieval techniques [45] leverage social tags to optimize performance without GPU dependency. Enterprise search systems utilize declarative frameworks like PyTerrier [43] to dynamically adapt retrieval pipelines across heterogeneous corporate data repositories.

VI. CONCLUSION

In this work, we conducted a systematic review focused on optimization techniques applied to Information Retrieval (IR) algorithms to assess their impact on efficiency and performance. The results suggest that, while optimization techniques can significantly enhance response time, memory efficiency, and scalability, achieving optimal performance in large-scale IR systems remains challenging. Computational costs and the complexity of implementing advanced techniques can hinder the scalability of IR solutions, especially in distributed systems where resource distribution and coordination add additional layers of complexity.

Despite continuous efforts in optimization research, certain technical limitations persist. Many studies indicate ongoing difficulties in balancing response time reduction with memory efficiency, especially in large-scale environments where data volume places additional strain on computational resources. Furthermore, adopting parallel architectures, such as GPUs, requires extensive modifications to codebases and hardware infrastructure, which can impede implementation at scale and may not be feasible in all environments. These challenges underscore the importance of adaptable optimization strategies that can address the balance of performance gains and resource constraints while considering the evolving nature of IR applications.

ACKNOWLEDGMENT

The authors acknowledge the use of generative AI tools in the preparation of this manuscript. Specifically, Claude (Anthropic), Gemini (Google), and ChatGPT (OpenAI) were employed to assist with grammar checking, text refinement, literature review organization, and clarity improvements throughout the manuscript. These AI tools were used as writing assistants to enhance the presentation of them original research, ideas, and analysis. All scientific content, methodology, analysis, conclusions, and intellectual contributions remain entirely the work of them, who take full responsibility for the accuracy and integrity of the manuscript.

REFERENCES

[1] S. Buttcher, C. L. Clarke, and G. V. Cormack, *Information Retrieval: Implementing and Evaluating Search Engines*. Cambridge, MA, USA: MIT Press, 2016.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[3] T. Shaikh, "Comparing performance of various optimization algorithms for effective information retrieval—A review," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 5, no. 9, pp. 747–750, Nov. 2017.

[4] D. Reinsel, J. Gantz, and J. Rydning, "The digitization of the world: From edge to core," *IDC White Paper*, vol. 2, no. 3, pp. 1–24, 2020.

[5] A. Yazdanbakhsh, B. Thwaites, H. Esmaeilzadeh, G. Pekhimenko, O. Mutlu, and T. C. Mowry, "Mitigating the memory bottleneck with approximate load value prediction," *IEEE Design Test*, vol. 33, no. 1, pp. 32–42, Feb. 2016.

[6] A. Carrera-Rivera, W. Ochoa, F. Larrinaga, and G. Lasa, "How-to conduct a systematic literature review: A quick guide for computer science research," *MethodsX*, vol. 9, Jan. 2022, Art. no. 101895.

[7] B. Mitra and N. Craswell, "An introduction to neural information retrieval," *Found. Trends Inf. Retr.*, vol. 13, no. 1, pp. 1–126, 2018.

[8] L. P. Valem, D. C. G. Pedronette, R. D. S. Torres, E. Borin, and J. Almeida, "Effective, efficient, and scalable unsupervised distance learning in image retrieval tasks," in *Proc. 5th ACM Int. Conf. Multimedia Retr.*, Jun. 2015, pp. 51–58.

[9] L. P. Valem and D. C. G. Pedronette, "Unsupervised similarity learning through Cartesian product of ranking references for image retrieval tasks," in *Proc. 29th SIBGRAPI Conf. Graph., Patterns Images (SIBGRAPI)*, Oct. 2016, pp. 249–256.

[10] A. Z. Broder, D. Carmel, M. Herscovici, A. Soffer, and J. Zien, "Efficient query evaluation using a two-level retrieval process," in *Proc. 12th Int. Conf. Inf. Knowl. Manage.*, Nov. 2003, pp. 426–434.

[11] C. M. Daoud, E. S. de Moura, D. Fernandes, A. S. da Silva, C. Rossi, and A. Carvalho, "Waves: A fast multi-tier top-k query processing algorithm," *Inf. Retr. J.*, vol. 20, no. 3, pp. 292–316, Jun. 2017.

[12] J. Guo, Y. Fan, L. Pang, L. Yang, Q. Ai, H. Zamani, C. Wu, W. B. Croft, and X. Cheng, "A deep look into neural ranking models for information retrieval," *Inf. Process. Manage.*, vol. 57, no. 6, Nov. 2020, Art. no. 102067.

[13] A. Yates, R. Nogueira, and J. Lin, "Pretrained transformers for text ranking: BERT and beyond," in *Proc. 14th ACM Int. Conf. Web Search Data Mining*, Mar. 2021, pp. 1154–1156.

[14] J. Wang, J. X. Huang, X. Tu, J. Wang, A. J. Huang, M. T. R. Laskar, and A. Bhuiyan, "Utilizing BERT for information retrieval: Survey, applications, resources, and challenges," *ACM Comput. Surv.*, vol. 56, no. 7, pp. 1–33, Jul. 2024.

[15] W. X. Zhao, J. Liu, R. Ren, and J.-R. Wen, "Dense text retrieval based on pretrained language models: A survey," *ACM Trans. Inf. Syst.*, vol. 42, no. 4, pp. 1–60, Jul. 2024.

[16] G. Peikos and G. Pasi, "A systematic review of multidimensional relevance estimation in information retrieval," *WIREs Data Mining Knowl. Discovery*, vol. 14, no. 5, p. 1541, Sep. 2024.

[17] F. A. Ruambo and M. R. Nicholaus, "Towards enhancing information retrieval systems: A brief survey of strategies and challenges," in *Proc. 11th Int. Congr. Ultra Modern Telecommun. Control Syst. Workshops (ICUMT)*, Oct. 2019, pp. 1–8.

[18] W. Ke, *Scalability of Findability: Decentralized Search and Retrieval in Large Information Networks*. Chapel Hill, NC, USA: Univ. North Carolina Chapel Hill, 2010.

[19] Z. Lu, *Scalable Distributed Architectures for Information Retrieval*. Amherst, MA, USA: Univ. Massachusetts Amherst, 1999.

[20] D. Hawking, A. Moffat, and A. Trotman, "Efficiency in information retrieval: Introduction to special issue," *Inf. Retr. J.*, vol. 20, no. 3, pp. 169–171, Jun. 2017.

[21] K. A. Hambarde and H. Proença, "Information retrieval: Recent advances and beyond," *IEEE Access*, vol. 11, pp. 76581–76604, 2023.

[22] N. Tonellotto, C. Macdonald, and I. Ounis, "Efficient query processing for search systems," *Found. Trends Inf. Retr.*, vol. 14, no. 3, pp. 241–329, 2020.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[24] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Comput. Surv.*, vol. 54, no. 10s, pp. 1–41, 2022.

[25] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, and J.-R. Wen, "Large language models for information retrieval: A survey," 2023, *arXiv:2308.07107*.

[26] K. Shah, S. Kopru, and J. D. Ruvini, "Neural network based extreme classification and similarity models for product matching," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol. (Industry Papers)*, 2018, pp. 8–15.

[27] O. Shahmirzadi, A. Lugowski, and K. Younge, "Text similarity in vector space models: A comparative study," in *Proc. 18th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2019, pp. 659–666.

[28] M. Fazlali, M. Mirhosseini, M. Shahsavari, A. Shafarenko, and M. Mashinchi, "GPU-based parallel technique for solving the N-similarity problem in textual data mining," in *Proc. 3rd Int. Conf. Distrib. Comput. High Perform. Comput. (DCHPC)*, May 2024, pp. 1–6.

[29] E. Dritsas, M. Trigka, G. Vonitsanos, A. Kanavos, and P. Mylonas, "An apache spark implementation for text document clustering," in *Proc. 17th Int. Workshop Semantic Social Media Adaptation Personalization (SMAP)*, Nov. 2022, pp. 1–6.

[30] C. Topcuoğlu, K. Kaya, and E. Savaş, "A generic private information retrieval scheme with parallel multi-exponentiations on multicore processors," *Concurrency Comput., Pract. Exper.*, vol. 30, no. 21, Nov. 2018, Art. no. e4685.

[31] Y. Liu, J. Wang, and S. Swanson, "Griffin: Uniting CPU and GPU in information retrieval systems for intra-query parallelism," *ACM SIGPLAN Notices*, vol. 53, no. 1, pp. 327–337, Mar. 2018.

[32] J. Zhang, A. Naruse, X. Li, and Y. Wang, "Parallel top-K algorithms on GPU: A comprehensive study and new methods," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2023, pp. 1–13.

[33] C. Wei, L. Qingbo, D. E. Na, and C. Congli, "A novel redundant data retrieval model based on parallel batch algorithm," in *Proc. Int. Conf. Inventive Comput. Technol. (ICICT)*, Apr. 2023, pp. 518–522.

[34] C. Guo, H. Chen, F. Zhang, and C. Li, "Distributed join algorithms on multi-CPU clusters with GPUDirect RDMA," in *Proc. 48th Int. Conf. Parallel Process.*, Aug. 2019, pp. 1–10.

[35] P. Steinert, S. Wagenpfeil, P. Mc Kevitt, I. Frommholz, and M. Hemmje, "Parallelization strategies for graph-code-based similarity search," *Big Data Cognit. Comput.*, vol. 7, no. 2, p. 70, Apr. 2023.

[36] M. Lam, J. Johnson, W. Xiong, K. Maeng, U. Gupta, Y. Li, L. Lai, I. Leontiadis, M. Rhu, H.-H.-S. Lee, V. J. Reddi, G.-Y. Wei, D. Brooks, and E. Suh, "GPU-based private information retrieval for on-device machine learning inference," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, vol. 1, Apr. 2024, pp. 197–214.

[37] Y. Shan, J. Jiao, J. Zhu, and J. Mao, "Recurrent binary embedding for GPU-enabled exhaustive retrieval from billion-scale semantic vectors," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 2170–2179.

[38] D. Zheng, X. Song, C. Ma, Z. Tan, Z. Ye, J. Dong, H. Xiong, Z. Zhang, and G. Karypis, "DGL-KE: Training knowledge graph embeddings at scale," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2020, pp. 739–748.

[39] D. Wang, L. Liu, and Y. Liu, "Normalized storage model construction and query optimization of book multi-source heterogeneous massive data," *IEEE Access*, vol. 11, pp. 96543–96553, 2023.

[40] C. Ni, "Top-k query optimization on the hierarchical memory structure," in *Proc. IEEE 6th Int. Conf. Autom., Electron. Electr. Eng. (AUTEEE)*, Dec. 2023, pp. 1075–1080.

[41] S. Hofstätter, N. Rekabsaz, C. Eickhoff, and A. Hanbury, "On the effect of low-frequency terms on neural-IR models," in *Proc. 42nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2019, pp. 1137–1140.

[42] S.-C. Lin, A. Ahmad, and J. Lin, "MAggretriever: A simple yet effective approach to zero-shot multilingual dense retrieval," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 11688–11696.

[43] C. Macdonald and N. Tonellotto, "Declarative experimentation in information retrieval using PyTerrier," in *Proc. ACM SIGIR Int. Conf. Theory Inf. Retr.*, Sep. 2020, pp. 161–168.

[44] F. Pisani, L. P. Valem, D. C. G. Pedronette, R. da S. Torres, E. Borin, and M. Breternitz, "A unified model for accelerating unsupervised iterative re-ranking algorithms," *Concurrency Comput., Pract. Exper.*, vol. 32, no. 14, Jul. 2020, Art. no. e5702.

[45] J. He, S. Wang, Q. Qu, W. Zhang, and Q. Huang, "Efficient cross-modal retrieval using social tag information towards mobile applications," in *Mobility Analytics for Spatio-Temporal and Social Data*, vol. 10731. Cham, Switzerland: Springer, 2018, pp. 157–176.

[46] D. Cohen, J. Foley, H. Zamani, J. Allan, and W. B. Croft, "Universal approximation functions for fast learning to rank: Replacing expensive regression forests with simple feed-forward networks," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jun. 2018, pp. 1017–1020.

[47] S. Shrestha, N. Reddy, and Z. Li, "ESPN: Memory-efficient multi-vector information retrieval," in *Proc. ACM SIGPLAN Int. Symp. Memory Manage.*, Jun. 2024, pp. 95–107.

[48] A. Atadoga, O. Chimezie Obi, F. Osasona, S. Onwusinkwue, S. Onwusinkwue, A. Ifesinachi Daraojimba, and S. Onimisi Dawodu, "Quantum computing in big data analytics: A comprehensive review: Assessing the advancements, challenges, and potential implications of quantum approaches in handling massive data sets," *Comput. Sci. IT Res. J.*, vol. 5, no. 2, pp. 498–517, Feb. 2024.

[49] E. P. Frady, G. Orchard, D. Florey, N. Imam, R. Liu, J. Mishra, J. Tse, A. Wild, F. T. Sommer, and M. Davies, "Neuromorphic nearest neighbor search using Intel's pohoiki springs," in *Proc. Neuro-Inspired Comput. Elements Workshop*, Mar. 2020, pp. 1–10.

[50] C. Tang, C. Wang, M. Lu, and P. Jin, "An efficient in-memory analytics system based on persistent memory," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2022, pp. 6818–6820.

[51] S. S. Pathak and D. Rao, "Adaptive system for handling variety in big text," in *Intelligent Communication and Computational Technologies: Proceedings of Internet of Things for Technological Development, IoT4TD 2017*. Cham, Switzerland: Springer, 2017, pp. 305–313.

[52] A. Xain, A. Goyal, B. Singh, and S. Sharma, "Multilinguistic approach towards information retrieval system for big data," in *Proc. 3rd Int. Conf. Intell. Sustain. Syst. (ICISS)*, Dec. 2020, pp. 159–164.

[53] M. Yan, H. Huang, Y. Liu, J. Zhao, X. Gao, C. Xu, Z. Guan, and W. Zhao, "TruthSR: Trustworthy sequential recommender systems via user-generated multimodal content," 2024, *arXiv:2404.17238*.

[54] K. Balog, "Multimodal information retrieval: A survey of the state of the art," *Found. Trends Inf. Retr.*, vol. 14, no. 5, pp. 401–533, 2020.

[55] Z. Wang and C. Zhan, "Adaptive resource allocation for large-scale search systems," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2021, pp. 1379–1389.

[56] J. Chen and Y. Liu, "Dynamic resource allocation for distributed search systems," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 7, pp. 3210–3224, Jan. 2022.

[57] K. K. Agbele, E. F. Ayetiran, K. D. Aruleba, and D. O. Ekong, "Algorithm for information retrieval optimization," in *Proc. IEEE 7th Annu. Inf. Technol., Electron. Mobile Commun. Conf. (IEMCON)*, Oct. 2016, pp. 1–8.

[58] S. Khan, M. Asjad, and A. Ahmad, "Review of modern optimization techniques," *Int. J. Eng. Res.*, vol. V4, no. 4, pp. 181–2278, Apr. 2015.

[59] L. Mu, L. Chen, and F. Wang, "A heuristic for legacy information system re-modularization," in *Proc. Int. Conf. Appl. Techn. Cyber Secur. Intell. ATCI Appl. Techn. Cyber Secur. Intell.*, 2018, pp. 505–516.

[60] Z. Jia and Z. Zhang, "Dissecting the gpu architecture for deep learning," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2018, pp. 1–12.

[61] D. Mikami, "Massively parallel processing of deep neural networks," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, Sep. 2021, pp. 941–950.

[62] A. Rashed and B. B. Cambazoglu, "Cost-aware cloud-based search systems," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2020, pp. 1401–1410.
[63] Y. Diao and Y. Li, "Big data as a service: Issues and challenges," *IEEE Trans. Services Comput.*, vol. 13, no. 5, pp. 834–847, Jan. 2020.
[64] G. Zuccon and B. Koopman, "Search engine optimization: A study of the economic impact of algorithmic changes," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2021, pp. 2059–2063.
[65] Z. Li, Z. He, and L. Wu, "Learning to cache for search engines," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2022, pp. 2105–2116.
[66] M. Poddar, T. Dey, C. Zhai, and J. Allan, "Understanding and improving cache performance in modern search engines," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2021, pp. 1268–1278.

ANDRÉ TOMITAN BOCCES received the B.S. degree in computer science from São Paulo State University (UNESP), Rio Claro, Brazil, in 2023, where he is currently pursuing the M.S. degree in computer systems.

His research focuses on optimization techniques for information retrieval methods, memory management architectures, and high-performance computing. His research interests include the optimization and refactoring of large-scale retrieval pipelines, and parallel and low-level programming.

ALEXANDRO BALDASSIN received the Ph.D. degree in computer science from UNICAMP, in 2009. He completed a Postdoctoral Fellowship with Instituto Superior Técnico/INESC-ID, Lisbon, from 2019 to 2020. He is currently an Associate Professor (MS-5.3) with the Institute of Geosciences and Exact Sciences (IGCE), São Paulo State University (UNESP), Rio Claro, Brazil. His research focuses on parallel systems, particularly hardware-software interaction and infrastructure. He has worked at Microsoft Research, in 2010, under Prof. James Larus, holding a registered patent, and was a Visiting Professor with Bangor University, U.K., in 2016, and the University of Nottingham Ningbo China, in 2019, collaborating with researchers from Alberta and Washington. He actively promotes high-performance computing (HPC) in Brazil, having coordinated São Paulo Regional School on HPC (ERAD-SP), in 2024, and chaired São Paulo Regional Council on HPC (CRAD-SP), since 2023. He is also a Regular Program Committee Member of the Symposium on High-Performance Computer Systems (SSCAD), where he served as the Program Chair, in 2022.

DANIEL CARLOS GUIMARÃES PEDRONETTE received the Ph.D. degree in computer science from UNICAMP, in 2012, and the Habilitation degree in digital image processing from São Paulo State University (UNESP), Rio Claro, Brazil, in 2019.

He is currently an Associate Professor (MS-5.3) with the Department of Statistics, Applied Mathematics, and Computation (DEMAC), UNESP. He has been a Visiting Scholar with Bangor University, U.K., in 2016, supported by the Royal Academy of Engineering, and Temple University, USA, from 2019 to 2020, supported by the Fulbright Commission. He has authored more than 140 scientific papers and received multiple best paper awards and nominations at national and international conferences, including SIBGRAPI, ICIP, and HPCS. His research interests include information retrieval, image processing and analysis, machine learning, and high-performance computing.

Dr. Pedronette is a CNPq Research Productivity Fellow (Level 2) and the Vice-Leader of the Center for Applied Natural Sciences (UNESPetro). He serves as an Associate Editor for *Pattern Recognition* (Elsevier) and has been the Vice-Coordinator of the Graduate Program in Computer Science at UNESP, since 2021.

ALLBERSON BRUNO DE OLIVEIRA DANTAS (Member, IEEE) received the bachelor's and master's degrees in high-performance computing from the Federal University of Cearà (UFC), the Postgraduate degree in project management from Cruzeiro do Sul University, in 2015, and the Ph.D. degree in computer science from UFC, in 2017.

He has more than ten years of experience in data science and more than five years in research and development projects between universities and industry, building solutions in computer vision, natural language processing, big data, and large graph processing. He worked for ten years as a Systems Analyst with Serpro and has been teaching undergraduate and postgraduate courses in data science, web programming, and distributed systems, since 2011. He co-authored several papers in international journals and conferences, such as *Concurrency and Computation: Practice and Experience*, *Science of Computer Programming*, and the International Conference on Computer and Information Science (Springer), and contributed to initiatives, such as BigData Fortaleza for AI-based urban policy development. His research interests include high-performance computing, machine and deep learning, and cloud computing, with an emphasis on the formal verification and large-scale parallel processing of intelligent systems.

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614